

Guía: amissv.efactusv.com en un VPS OVH nuevo, desplegado con Cetmix Tower

Objetivo: reproducir **tal cual** el stack de `efactusv-odoo-deploy` (odoo build + postgres:15 + mailpit + nginx en contenedor + certbot en contenedor) en un servidor OVH nuevo, pero sin tocar el servidor a mano: Tower crea archivos, clona addons, construye, inicializa la BD, crea el DNS en OVH, emite el SSL y deja el jet en `running`.

Archivo que lo hace posible: `docs/tower/efactusv_odoo18_dedicado.yaml` (70 registros, importación validada en BD local).

0. Vocabulario (lo mínimo para no perderse)

Término Tower	Traducción a tu mundo	Ejemplo en esta guía
Server	Un VPS al que Tower entra por SSH	El VPS OVH nuevo de amissv
Server Template	Plantilla para dar de alta servidores iguales (usuario, puerto, plan inicial)	<code>efactusv - VPS OVH Debian 12</code>
Key / Secret	Contraseñas y llaves. Se escriben <code>#!cxtower.secret.NOMBRE!</code> y Tower las sustituye y las oculta en los logs	<code>EFACTUSV_ODOO_DB_PASSWORD</code>
Variable	Valor con nombre <code>{{ nombre }}</code> . Se resuelve en cascada: jet → server → global	<code>instance_name = amissv</code>
Command	Una unidad de trabajo. Tipos: <i>SSH</i> (corre en el VPS), <i>Python</i> (corre en tu Odoo madre), <i>File</i> (sube un archivo renderizado), <i>Plan</i> (llama otro plan)	<code>docker compose build</code>
File Template	Un archivo con variables que Tower rellena y sube por SFTP	<code>docker-compose.yml</code> , <code>odoo.conf</code>
Flight Plan	Lista ordenada de comandos, tipo playbook. Si uno falla, para	Crear instancia (0 -> producción)
Jet	Una instancia de cliente , con estado (<code>running</code> , <code>stopped</code> ...) y ciclo de vida	El jet "amissv"
Jet Template	El molde: qué acciones tiene un jet, qué plan ejecuta cada acción, variables por defecto	<code>efactusv Odoo 18 dedicado</code>
Jet Action	Transición de estado: <i>desde</i> → <i>transitorio</i> → <i>hacia</i> , ejecutando un plan	Create: ∅ → starting → running
Scheduled Task	Cron de Tower (comando o plan cada X tiempo)	Backup diario
Server Log	Ventana de log que soporte puede ver sin SSH	<code>docker logs amissv-odoo</code>
YAML	Export/import de todo lo anterior. Los secretos no viajan , solo su nombre	El archivo de esta guía

Regla de oro: **Server = el hierro. Jet = lo que le cobras al cliente.**

1. Qué hace el plan "Crear instancia" (mapa mental)

Tu Odoo madre (Cetmix Tower)	VPS OVH (amissv)
10 mkdir	—SSH→ <code>/opt/efactusv/amissv/{config,nginx/conf.d,nginx/webroot,addons}</code>
20-60 subir 5 archivos	—SFTP→ <code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>config/odoo.conf</code> , <code>nginx/conf.d/default.conf</code> (HTTP), <code>nginx/ssl.conf.template</code>
70 clonar addons (token GitHub)	—SSH→ <code>addons/efactusv</code> (= el submodule del proyecto viejo)
80 docker compose build	—SSH→ <code>imagen odoo:18.0 + pycryptodome xlrd xlswriter</code>
90 up db + pg_isready	—SSH→ <code>amissv-db</code>
100 odoo -i {{ initial_modules }}	—SSH→ BD "amissv" creada (se salta si ya existe)
110 odoo shell: admin + web.base.url	—SSH→ <code>admin = admin@efactusv.com / secreto</code>
120 docker compose up -d	—SSH→ <code>amissv-odoo</code> , <code>amissv-mailpit</code> , <code>amissv-nginx</code>
130 OVH API (Python, en Odoo)	—API→ registro A <code>amissv.efactusv.com</code> → IP del VPS
140 esperar DNS (dig)	—SSH→
150 certbot + activar ssl.conf	—SSH→ <code>= setup-ssl.sh</code>
160 curl https://.../web/login	—SSH→
170 guardar URL en el jet	(Python)

Diferencias deliberadas respecto al `efactusv-odoo-deploy` manual (todas mejoran, ninguna cambia la arquitectura):

- Contraseñas (`odoo/odoo`, `admin_passwd` en claro en git) → **secretos de Tower**.
- `db_name = False + dbfilter = .*` → `db_name = amissv`, `dbfilter = ^amissv$`, `list_db = False` (Tower crea la BD; el selector de BD público queda cerrado).
- Sin `logfile` → los logs van a `stdout` y se ven con `docker logs / Server Log` de Tower.
- `nginx`: añadido `location /websocket` (Odoo 17/18 lo necesita; `/longpolling` se mantiene por compatibilidad).
- Mailpit publicado solo en `127.0.0.1:8025` (antes estaba abierto a Internet sin autenticación).
- `certbot` con `--keep-until-expiring` (re-ejecutar el plan no gasta la cuota de Let's Encrypt).
- El submodule git se sustituye por `git clone --depth 1` con un token de solo lectura **queno se guarda en disco**.

2. Preparar el Odoo madre (una sola vez)

2.1 Módulos a instalar (en este orden)

Módulo	Para qué
<code>cetmix_tower_server</code>	Núcleo (arrastra <code>rpc_helper</code> , <code>web_notify</code>)
<code>cetmix_tower_yaml</code>	Importar el YAML de esta guía
<code>cetmix_tower_ovh</code>	Inyecta el cliente <code>ovh</code> en comandos Python (DNS)
<code>cetmix_tower_server_queue</code> + <code>queue_job</code>	Recomendado: los planes largos (<code>build</code> , <code>certbot</code>) corren en cola y no bloquean HTTP
<code>efactusv_tower_contract</code>	Botón Create Instance en el contrato OCA + parar/arrancar al terminar/reactivar
<code>cetmix_tower_git</code>	Opcional. Modela repos y genera <code>repos.yaml</code> de git-aggregator. Para este stack (un solo repo) no hace falta

```
pip install -r efactusv-cetmix-tower/requirements.txt # paramiko<4, pyyaml, ovh...
./odoo-bin -c odoo.conf -d <bd_madre> \
-i cetmix_tower_server,cetmix_tower_yaml,cetmix_tower_ovh,cetmix_tower_server_queue,efactusv_tower_contract \
--stop-after-init
```

`odoo.conf` del madre: `workers >= 2`, `server_wide_modules = base,web,queue_job` (si usas la cola), `limit_time_real = 1200` (el `docker compose build` tarda). En el `nginx` del madre proxy de `/websocket` a `8072`, si no los comandos parecen colgados aunque terminen.

2.2 Permisos

Ajustes > Usuarios > tu usuario > pestaña *Cetmix Tower*: grupo **Root** y *Access Level* **Root** (el plan de destruir y el borrado de DNS son nivel root).

2.3 Llave SSH maestra de Tower

```
ssh-keygen -t ed25519 -C "cetmix-tower@efactusv" -f ~/.ssh/tower_efactusv -N ""
```

Tower > Settings > Keys and Secrets > New: Key Type = **SSH Key**, referencia `SSH_TOWER_EFACTUSV`, pegar la **privada** completa. Guarda la pública para el paso 3.

2.4 Credenciales OVH (API)

1. Ir a `https://eu.api.ovh.com/createToken/` con tu cuenta OVH.
2. Rights: `GET /domain/zone/*`, `POST /domain/zone/*`, `PUT /domain/zone/*`, `DELETE /domain/zone/*`. Para crear servidores desde Tower (opción B de la sección 3) añade también `GET /cloud/*`, `POST /cloud/*`, `DELETE /cloud/*`.
3. Te da Application Key, Application Secret y Consumer Key.

2.5 Token de GitHub

GitHub > Settings > Developer settings > Fine-grained tokens > repo `porti1876/efactusv`, permiso **Contents: Read-only**. (Los repos son privados; sin esto el clone falla.)

2.6 Crear los secretos (antes de importar el YAML)

Tower > Settings > Keys and Secrets, Key Type = **Secret**, exactamente estas referencias:

Referencia	Valor
EFACTUSV_OD00_DB_PASSWORD	contraseña Postgres de la instancia (genera una larga)
EFACTUSV_OD00_ADMIN_PASSWORD	admin_passwd y contraseña del usuario admin
EFACTUSV_GITHUB_TOKEN	el PAT de 2.5
OVH_APPLICATION_KEY / OVH_APPLICATION_SECRET / OVH_CONSUMER_KEY	los de 2.4

El YAML también crea estos registros vacíos si no existen; si ya existen, el import los deja igual (usa *Update existing record* y el valor no se toca porque no viaja en el YAML).

Multi-cliente: cada secreto puede tener **valor por jet** (pestaña *Secret Values* del secreto o del jet). Así EFACTUSV_OD00_DB_PASSWORD es distinto para amissv y para el siguiente cliente con el mismo blueprint.

2.7 Importar el YAML

Tower > Settings > YAML > **Import** (o el botón *Import YAML* de cualquier lista) > subir docs/tower/efactusv_odoo18_dedicado.yaml > *If record exists* = **Update** > Import.

Qué aparece:

- 6 secretos (vacíos si no existían), 22 variables.
- 5 file templates (Files > Templates).
- 26 comandos, 7 flight plans (Commands).
- Server templates efactusv - VPS OVH Debian 12 (Docker) (VPS clásico) y efactusv - OVH Public Cloud Debian 12 (creado por API).
- Jet template efactusv odoo 18 dedicado (docker nginx+certbot) con acciones Create / Stop / Start / Destroy, server log "Logs Odoo" y tarea programada "Backup diario".

Ajustes manuales tras importar (campos que el YAML no transporta):

- Server template > **SSH Private Key** = SSH_TOWER_EFACTUSV.
- Jet template > *State on Contract Termination* = **Stopped**, *State on Contract Reactivation* = **Running** (campos de efactusv_tower_contract).
- Settings > General > **Command Timeout** = 1800 s (el build de la imagen y certbot pueden tardar).

Re-importar el mismo YAML con *Update existing record* falla en las plantillas de servidor duplicate key ... unique_variable_value_template: limitación del import de variable_value_ids). Para actualizar, importa con **Skip** los registros que no cambiaron, o borra antes la plantilla de servidor.

3. El VPS en OVH — dos formas

¿"Hetzner lo hace y OVH no"? Ni en este repo ni en el upstream de Cetmix existe un módulo Hetzner; cetmix_tower_aws y cetmix_tower_ovh solo inyectan la librería boto3 / ovh en comandos Python. Crear el servidor es siempre un comando Python que escribes tú. Con OVH **sí se puede**: el producto que se crea con una llamada a la API es **Public Cloud** (POST /cloud/project/{id}/instance). El rango "VPS" clásico se compra por carrito (/order/cart) y llega por email: no vale para automatizar.

Opción A — VPS clásico creado en el panel (manual, 5 min)

A.1 Crear el VPS: VPS > Order > Debian 12 > mínimo 2 vCPU / 4 GB / 80 GB (la config limit_memory_hard = 7.6 GB del odoo.conf sugiere 8 GB). Apunta la **IPv4**.

A.2 Bootstrap como root (lo único manual):

```
ssh root@<IP_VPS>
adduser --disabled-password --gecos "" deploy
usermod -aG sudo deploy
echo "deploy ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/deploy
mkdir -p /home/deploy/.ssh
echo 'ssh-ed25519 AAAA... cetmix-tower@efactusv' >> /home/deploy/.ssh/authorized_keys # pública de 2.3
chmod 700 /home/deploy/.ssh && chmod 600 /home/deploy/.ssh/authorized_keys
chown -R deploy:deploy /home/deploy/.ssh
```

Firewall OVH (o ufw): abrir **22, 80, 443** y nada más.

A.3 Alta en Tower: Servers > Templates > efactusv - VPS OVH Debian 12 (Docker) > Create Server:

Campo	Valor
Name	OVH-AMISSV-01
IPv4	la del VPS
Partner	el cliente amissv
(heredado) SSH user / port / auth / sudo	deploy / 22 / Key / Without password
Variables	base_dir=/opt/efactusv, ovh_endpoint=ovh-eu, ovh_zone_name=efactusv.com

Al crearlo, Tower ejecuta el plan **Preparar servidor (Docker)** (= tu install.sh, idempotente). Luego: **Show host key > Test Connection** (SSH + SFTP verde).

Opción B — Public Cloud creado desde Tower (100 % automático)

Lo que hace el comando efactusv - OVH Cloud: crear instancia y registrarla en Tower (plan efactusv - OVH Cloud: aprovisionar VPS nuevo):

1. Registra la clave pública de Tower en el proyecto Public Cloud (si no está)
2. Busca flavor e imagen por nombre en la región (b3-8 / "Debian 12")
3. POST /cloud/project/{id}/instance (si ya existe con ese nombre, la reutiliza)
4. Espera status ACTIVE + IPv4 pública (máx. 10 min)
5. create_server_from_template(...) en Tower (o actualiza la IP si el server ya existe)
6. Espera a que SSH responda, guarda el host key (sin "Show host key" manual)
7. Lanza el plan "Preparar servidor (Docker)"

La imagen Debian 12 de OVH ya trae el usuario `debian` con sudo NOPASSWD y la clave inyectada, por eso no hay bootstrap. La plantilla de servidor que usa es efactusv - OVH Public Cloud Debian 12 (usuario debian).

B.1 Requisitos

- Proyecto Public Cloud en OVH (Public Cloud > Create project). Su ID (serviceName) sale en la URL del panel o con GET /cloud/project.
- Token OVH con rights adicionales: GET /cloud/*, POST /cloud/*, DELETE /cloud/*.
- Variables globales (Settings > Variables > pestaña Values, o en el servidor desde el que ejecutes):

Variable	Valor
ovh_cloud_project	ID del proyecto
ovh_cloud_region	BHS5 (Canadá, la más cercana a El Salvador) o GRA11
ovh_cloud_flavor	b3-8 (2 vCPU / 8 GB). Ver GET /cloud/project/{id}/flavor?region=BHS5
ovh_cloud_image	Debian 12 (nombre exacto)
tower_ssh_public_key	contenido de ~/.ssh/tower_efactusv.pub
tower_ssh_key_ref	SSH_TOWER_EFACTUSV
ovh_server_template	efactusv_ovh_cloud_debian12

B.2 Ejecutar: abre cualquier servidor ya registrado (el comando ignora ese servidor) > Run Flight Plan > efactusv - OVH Cloud: aprovisionar VPS nuevo > new_server_name = OVH-AMISSV-01 > Run. Con cetmix_tower_server_queue corre en segundo plano; sin cola sube limit_time_real del madre a ≥ 1200. Al terminar verás el servidor OVH-AMISSV-01 con IP, host key y Docker instalado. Después abre el servidor y pon el **Partner** = amissv (el comando hereda el partner del servidor desde el que lo lanzas).

Para dar de baja el hierro: comando efactusv - OVH Cloud: BORRAR instancia (Root; borra en OVH y archiva el server en Tower).

B.3 Firewall: Public Cloud no filtra puertos por defecto (todo abierto). Añade al plan de preparación `ufw allow 22,80,443/tcp && ufw --force enable` o usa *Security Groups* del proyecto si quieres cerrar el resto.

4. Crear la instancia amissv

4.1 Vía contrato (recomendado, flujo real de negocio)

1. Contratos > contrato de amissv > pestaña Tower: **Instance Template** = `efactusv Odoo 18 dedicado`, **Instance Server** = `OVH-AMISSV-01`.
2. Botón **Create Instance** → se crea el jet enlazado al contrato y al partner.
3. Abrir el jet y rellenar variables (son las **requeridas** del template):

Variable	Valor
<code>instance_name</code>	<code>amissv</code>
<code>instance_domain</code>	<code>amissv.efactusv.com</code>
<code>db_name</code>	<code>amissv</code>
<code>initial_modules</code>	<code>base</code> (o <code>base,l10n_sv,...</code> si quieres inicializar con tus módulos)
<code>admin_email</code>	correo del admin de amissv (también se usa para Let's Encrypt)
<code>odoo_workers</code>	<code>2</code>

4. Botón **Actions** > **Create**. El jet pasa a `starting`; sigue el progreso en *Flight Plan Logs*. Al terminar: jet `running`, campo URL = `https://amissv.efactusv.com`.

4.2 Vía wizard (sin contrato)

Jets > **Launch New Jet** > template + servidor + nombre + mismas variables > luego Actions > Create.

4.3 Qué revisar si algo falla (fail-fast: el plan para en la línea rota)

Línea	Síntoma	Causa típica
70 addons	<code>Authentication failed</code>	token GitHub sin <i>Contents: read</i> o expirado
80 build	<code>timeout -206</code>	subir <i>Command Timeout</i> , o <code>docker compose build</code> a mano la 1ª vez
100 init	error de módulo	<code>initial_modules</code> con módulo inexistente; revisa <code>docker logs amissv-odoo</code>
130 DNS	403 OVH	rights del token OVH no incluyen <code>/domain/zone/*</code>
140 DNS wait	timeout	TTL viejo / registro previo apuntando a otra IP → espera y relanza
150 SSL	certbot falla	puerto 80 cerrado en firewall OVH o DNS aún no propagado

Relanzar = volver a ejecutar **Create** (todo es idempotente: no duplica BD, ni DNS, ni certificado). Si el jet se queda en `starting`, ábrelo > *Bring to state* > `stopped` y vuelve a lanzar.

5. Migrar los datos del servidor viejo (opcional)

El plan crea una BD limpia. Para traer la BD y filestore del servidor actual:

```
# servidor viejo
docker exec amissv-db pg_dump -U odoo -Fc <bd_vieja> > /tmp/amissv.dump
docker run --rm -v odoo_odoo-data:/data:ro -v /tmp:/backup alpine tar czf /backup/amissv-fs.tar.gz -C /data
.
scp /tmp/amissv.dump /tmp/amissv-fs.tar.gz deploy@<IP_NUEVA>:/opt/efactusv/backups/

# servidor nuevo (o como comando SSH en Tower)
cd /opt/efactusv/amissv
docker compose stop odoo
docker compose exec -T db psql -U odoo -d postgres -c "DROP DATABASE amissv" -c "CREATE DATABASE amissv OWNER odoo"

docker compose exec -T db pg_restore -U odoo -d amissv --no-owner < /opt/efactusv/backups/amissv.dump
docker run --rm -v amissv_odoo-data:/data -v /opt/efactusv/backups:/backup alpine
\
sh -c "rm -rf /data/filestore/amissv && mkdir -p /data/filestore && tar xzf /backup/amissv-fs.tar.gz -C /data"
docker compose up -d odoo
```

(Si la BD vieja tenía otro nombre, renombra la carpeta filestore/<nombre_viejo> a filestore/amissv.) Después lanza el comando **Configurar usuario admin** desde el jet para fijar web.base.url.

6. Operación diaria desde Tower

Necesito...	Dónde
Ver logs de Odoo sin SSH	Jet > Server Logs > <i>Logs Odoo</i> (nivel User: soporte puede)
Desplegar código nuevo	Jet > Run Flight Plan > Desplegar código (pide modules_to_update, ej. l10n_sv,efactusv_base)
Reiniciar Odoo	Jet > Run Command > <i>Reiniciar contenedor Odoo</i>
Parar / arrancar	Jet > Actions > Stop / Start
Backup manual	Jet > Run Command > <i>Backup BD + filestore</i> (el diario ya corre solo, retención 14 días en /opt/efactusv/backups)
Renovar SSL	Comando <i>Renovar certificado SSL</i> (o crea una Scheduled Task semanal con él)
Dar de baja	Contrato terminado → jet a stopped automáticamente. Destruir (Root) = pg_dump final + down -v + borrar carpeta + borrar DNS
Cambiar workers/imagen	Editar variable en el jet > relanzar <i>Subir docker-compose.yml + Subir odoo.conf + docker compose up -d</i>

Versionado: cualquier cambio que hagas en la UI → Settings > YAML > Export > committea en docs/tower/. Referencias = clave de emparejamiento, no las cambies.

7. Siguiendo cliente

Mismo YAML, nada que importar de nuevo:

1. VPS nuevo: opción A (panel + bootstrap) o B (plan OVH Cloud: *aprovisionar VPS nuevo*, sin tocar el servidor).
2. Create Server desde la plantilla.
3. Contrato > Create Instance > variables (instance_name=cliente2, instance_domain=cliente2.efactusv.com, db_name=cliente2) > Create.
4. Valor del secreto EFACTUSV_ODOO_DB_PASSWORD / _ADMIN_PASSWORD **por jet** para que cada cliente tenga el suyo.

limit_per_server = 1 en el template evita lanzar dos jets en el mismo VPS (nginx ocupa 80/443). Si algún día quieres varios clientes por servidor, el blueprint de efactusv_tower_odoo (nginx en el host + puertos 127.0.0.1:100xx) es el camino; este YAML es el "dedicado".

Notas encontradas al revisar tu código actual

- efactusv_tower_odoo/data/tower_blueprint.xml > comando *Configure Odoo administrator*: el odoo shell hace rollback al salir, así que sin env.cr.commit() **el cambio de contraseña no se guarda**. El comando equivalente de este YAML ya incluye el commit.
- efactusv-odoo-deploy/nginx/*.conf no proxean /websocket; en Odoo 18 el bus (chat, notificaciones, refresco de

vistas) no funciona sin ello.

- BD de pruebas local `test_tower_yaml` (copia de `test_tower_contract` + yaml + ovh) tiene este YAML ya importado si quieres verlo en la UI.